

COURSE PROJECT REPORT: CS203

Smart Transportation Network Planning and Resilience Analysis

Course Project on Graph Theory and Algorithmic Optimization

Submitted By:	Kanjula Sumanth Reddy
Roll Number:	240510
Program:	B.Tech, Computer Science and Engineering
Institution:	Indian Institute of Technology Kanpur (IITK)
Date of Submission:	July 5, 2026

1. Introduction & Network Modeling

In this project, we model and optimize a large-scale smart transportation network connecting **500 cities** using concepts from graph theory. The transportation network is modeled as a weighted, undirected graph $G = (V, E)$. The vertices V represent individual cities generated at random 2D coordinate positions (in a 1000x1000 grid). The edges E represent the physical road links between cities, and their weights represent the physical distances (in kilometers) computed using the Euclidean distance metric.

To build a realistic, sparse network rather than a complete graph, each node is initially connected to its k -nearest neighbors. To ensure that all cities are reachable, we run a Union-Find set to find disjoint connected components, and iteratively add edges between the closest pairs of vertices belonging to disconnected components. This guarantees a single connected component prior to running simulations.

2. Task 1: MST Optimization

Objective: Identify the minimum construction cost layout that keeps all 500 cities connected.

Implementation: We implemented **Kruskal's Algorithm** using a Disjoint Set Union (DSU) data structure with path compression. All candidate edges are sorted in ascending order of weight. We iterate through the sorted list, choosing edges that connect different connected components until we have selected exactly 499 roads. The time complexity of sorting edges is $O(E \log E)$.

Console Output:

```
Original Network Cost = 65914 km
Optimized MST Cost = 14493 km
Total Cost Saved = 78.0 %

Selected Roads (First 10):
Road 1: City 414 to City 435 (1 km)
Road 2: City 14 to City 137 (1 km)
Road 3: City 244 to City 336 (2 km)
[All 499 structural spans successfully written to path buffer]
```

3. Task 2: Strategic City Identification

Objective: Find the most critical transportation hubs to prioritize infrastructure projects.

Implementation: We compute the **Degree Centrality** of every vertex (representing the number of direct road connections sharing a city). The cities are ranked in descending order. Using the top candidates, the program recommends developmental roles: the rank 1 city is recommended for an *International Airport*, rank 2 for a *Logistics Hub*, and rank 3 for a *Railway Junction*. Time complexity is $O(V + E)$.

Console Output:

Top 10 Cities by Degree Centrality:

Rank 1: City 60 (Degree: 10)

Rank 2: City 195 (Degree: 10)

Rank 3: City 9 (Degree: 9)

Rank 4: City 10 (Degree: 9)

Top Candidates:

- Airport: City 60
- Logistics Hub: City 195
- Railway Junction: City 9

Recommendation: City 60 is a major hub and is recommended for an international airport.

4. Task 3: Disaster Recovery Routing

Objective: Model route adjustments and alternate paths when natural disasters destroy nodes or edges.

Implementation: We implemented **Dijkstra's Algorithm** using a heap-based priority queue (*std::priority_queue* in C++). When running routing queries, any nodes or edges that have been destroyed are explicitly ignored during neighbor updates. If a route does not exist due to partitioning, it prints the fallback string. The time complexity is $O((V + E) \log V)$.

Console Output:

```
[Demo Disruption Config]
Destroying Cities: 12, 55
Destroying Roads : 12-91, 44-67
Route Query: A = City 1, B = City 100 to D = City 203

Route from City 1 to City 203: Distance = 332 km
Path: City 1 → City 434 → City 71 → City 197 → City 306 → ... → City 203
Route from City 100 to City 203: Distance = 781 km
Path: City 100 → City 144 → City 390 → City 478 → ... → City 203
```

5. Task 4: Traffic-Aware Smart Routing

Objective: Compute paths that bypass congested roads by dynamically updating travel costs.

Implementation: We assign Low, Medium, or High traffic levels to roads. These correspond to weight multipliers of 1.0x, 1.5x, and 2.5x respectively. Dijkstra's algorithm computes the shortest path using these scaled weights ($w_{traffic} = w_{base} * multiplier$) to find traffic-avoiding paths, comparing the result to the normal free-flow path. The time complexity is $O((V + E) \log V)$.

Console Output:

```
[Demo Routing Query] Source: City 1 to Destination: City 250

Normal Route (no traffic):
Distance = 847 km
Path = City 1 → City 434 → City 231 → City 418 → ... → City 250

Traffic-Aware Route:
Distance = 904 km
Effective Travel Cost = 991.5 km (adjusted for traffic)
Path = City 1 → City 434 → City 231 → City 418 → City 90 → ... → City 250

Delay = 17.1%
```

6. Task 5: Critical Infrastructure Analysis

Objective: Diagnose single points of failure (cut-vertices and bridges) that partition the grid when removed.

Implementation: We write a connectivity analyzer that runs a **Breadth-First Search (BFS)** traversal on the graph. The analyzer systematically removes each node and edge one-by-one and counts the number of disconnected components created. The element that causes the maximum components is flagged as the most critical infrastructure. The time complexity is $O((V + E)^2)$.

Console Output:

```
Analyzing graph connectivity (500 nodes)... Please wait...
Base Component Count = 1

Critical Diagnostic Output:
Most Critical City: City 9 (Splits graph into 3 components)
Most Critical Road: City 1 - City 434 (Splits graph into 2 components)

[Removal of City 9 breaks network graph into 3 disconnected subgrids]
```

7. Summary & Conclusion

The project successfully implemented 5 tasks to model and optimize a smart transportation network with 500 cities. Using custom DSU and Dijkstra algorithms in standard C++, we demonstrate cost minimization, centrality hubs, disaster tolerance, congestion bypasses, and critical failure locations. The code compiles without external libraries and works efficiently.